

cgroup v2 leaf creation em um Docker-like system construído do zero com Python, Cython e C++

Estudo técnico sobre criação de folhas em cgroup v2, aplicação de limites de CPU, memória e PIDs, uso de *memory.max*, *pids.max* e *cpu.max*, regras top-down, e um plano de implementação orientado por System Design, Low Level Design, Domain-Driven Design e padrões GoF.

Escopo funcional. A feature analisada é a criação de folhas válidas em uma hierarquia unificada de cgroup v2 para workloads isolados, com limites hierárquicos e aderência às regras do kernel.

Arquitetura alvo. Control plane, metadados e API em Python; data plane e integração com o kernel em C++; Cython como fronteira estável entre os dois.

Base documental. Kernel Linux, man-pages, systemd, Docker Engine, runc, OCI runtime spec e libcontainer. ^{[1][3][4][8][9][10]}

Data. Março de 2026.

Linux

cgroup v2

systemd delegation

Docker semantics

Python + Cython + C++

Sumário

1. Problema e tese arquitetural	Definição do tema e posição do trabalho
2. Fundamentos do cgroup v2	Hierarquia unificada, leafs e regras top-down
3. CPU, memória e PIDs em profundidade	<i>cpu.max</i> , <i>memory.max</i> , <i>pids.max</i>
4. Docker original e a tradução para o projeto	Engine, containerd, runc, libcontainer e OCI
5. System Design da feature	Control plane, data plane, reconciliação e observabilidade
6. Low Level Design	Algoritmos, ordem de escrita, APIs e recovery
7. Domain-Driven Design	Bounded contexts, agregados, invariantes e eventos
8. Padrões de Design	Composite, Strategy, Template Method, Command, State e outros
9. Requisitos funcionais e não funcionais	Contrato mínimo para uma implementação séria
10. Roadmap de implementação e testes	Entrega incremental com redução de risco
11. Conclusão	Decisões recomendadas
Referências	Documentação oficial e material normativo

1. Problema e tese arquitetural

A feature pedida parece estreita à primeira vista, mas na prática toca o núcleo de qualquer sistema estilo Docker: quem controla a hierarquia de recursos, em que ordem escreve nos arquivos de cgroup, quando anexa processos, onde persiste o estado desejado e como evita que a modelagem de domínio seja contaminada por detalhes de syscall. Se essa base for ruim, o restante do runtime herdar inconsistência; se ela for boa, quase todo o resto se torna uma extensão previsível do mesmo contrato.

Em cgroup v2, a palavra “leaf” não deve ser tratada como mera convenção de nomenclatura. Ela designa, do ponto de vista prático, o nó terminal da subárvore em que o workload efetivamente vive. Os nós interiores distribuem controladores aos descendentes por meio de *cgroup.subtree_control*; o nó folha hospeda processos e recebe os arquivos concretos de limitação, como *memory.max*, *pids.max* e *cpu.max*. O kernel explicita que controladores não vêm habilitados por padrão, que a habilitação acontece no pai para afetar os filhos e que existem restrições top-down e a conhecida vedação de processos em nós interiores quando se trabalha com controladores de domínio. [1]

A tese arquitetural deste documento é simples e forte. O control plane em Python deve permanecer responsável pelo domínio: API, autenticação, persistência, políticas, normalização de limites, cálculo de *effective limits*, reconciliação e auditoria. O data plane em C++ deve se responsabilizar apenas pelo concreto: preparação de subárvore, criação da folha, aplicação ordenada dos arquivos de controle, *spawn*, anexação do PID ao cgroup, coleta de métricas e limpeza. Cython entra como fronteira explícita, fina e estável, não como um lugar para reescrever regra de negócio. Essa divisão preserva produtividade no topo e previsibilidade mecânica no fundo.

O contexto do Docker original reforça esse desenho. O Docker Engine expõe uma superfície cliente-servidor com daemon, API e CLI; delega o ciclo de vida de containers ao *containerd*; usa *runc* como runtime por padrão; e este, por sua vez, se apoia em *libcontainer* para materializar namespaces, capabilities e cgroups. O que no Docker aparece como uma flag de alto nível, como *--memory*, *--cpus* ou *--pids-limit*, desce até o kernel por uma cadeia de tradução que passa por um modelo canônico de recursos. [4][5][10]

Ponto central do projeto. A feature não é apenas “escrever três arquivos”. Ela é a capacidade de transformar um contrato declarativo de recursos em uma sequência válida, idempotente e recuperável de ações sobre uma hierarquia unificada de cgroup v2.



Figura 1 — Separação recomendada entre control plane, Cython e data plane nativo.

2. Fundamentos do cgroup v2

2.1. Hierarquia unificada, controladores e delegação

O cgroup v2 substitui a fragmentação do modelo v1 por uma hierarquia única montada tipicamente em `/sys/fs/cgroup`. Cada nó expõe metadados, como `cgroup.controllers` e `cgroup.subtree_control`, e, quando um controlador é habilitado no pai, seus arquivos de interface aparecem nos filhos. Essa semântica é decisiva: para aplicar `cpu.max`, `memory.max` e `pids.max` em uma folha, não basta criar o diretório da folha; é preciso garantir antes que o pai tenha distribuído os controladores corretos para baixo. [1]

O kernel também define a regra top-down. Um cgroup não raiz só pode habilitar um controlador para seus filhos se esse controlador já estiver disponível a partir do pai; simetricamente, um pai não deveria remover um controlador que ainda está em uso mais abaixo. Há, portanto, uma disciplina hierárquica que precisa aparecer na modelagem do software. Em um runtime sério, a árvore de cgroups não é um detalhe de armazenamento: ela é parte do domínio operacional e carrega invariantes. [1]

A delegação é o mecanismo que torna essa hierarquia compatível com ambientes geridos por systemd. A documentação do systemd é explícita sobre a regra do *single writer*: um nó da árvore não deve ter dois gerentes independentes escrevendo em paralelo em nomes, atributos ou filhos. Em hosts cujo PID 1 é o systemd, um runtime que tenta administrar arbitrariamente a raiz inteira por cgroupfs briga com o init do sistema. A forma correta é receber uma subárvore delegada ou usar diretamente o backend systemd. [3]

2.2. O que é “leaf creation” de fato

No contexto do projeto, *leaf creation* significa criar um nó terminal dedicado a um workload, de tal forma que esse nó possa hospedar processos e ao mesmo tempo respeite a política de distribuição de controladores definida pelos ancestrais. Uma folha não deveria ter filhos enquanto estiver hospedando processos, e um nó interior que distribui controladores de domínio para baixo não deveria conter processos de workload. Em outras palavras, a semântica correta é “interior para organizar e delegar; folha para executar e contabilizar”. [1]

Isso parece óbvio até aparecer o primeiro detalhe operacional sério. Se o seu próprio daemon estiver executando dentro da subárvore que pretende administrar, o nó que o contém deixa de ser um nó interior vazio. Em um host com systemd, o mais comum é que o serviço do runtime já nasça em um cgroup próprio. Se essa unidade receber `Delegate=`, ela ganha uma raiz delegada, mas essa raiz inicialmente contém o próprio processo do daemon. Antes de habilitar `+cpu`, `+memory` e `+pids` para os filhos, o runtime precisa criar uma folha interna, por exemplo `__runtime/`, mover a si mesmo para lá e só então deixar a raiz delegada vazia. Essa sutileza é uma das diferenças entre uma implementação didática e uma implementação correta. [1][3]

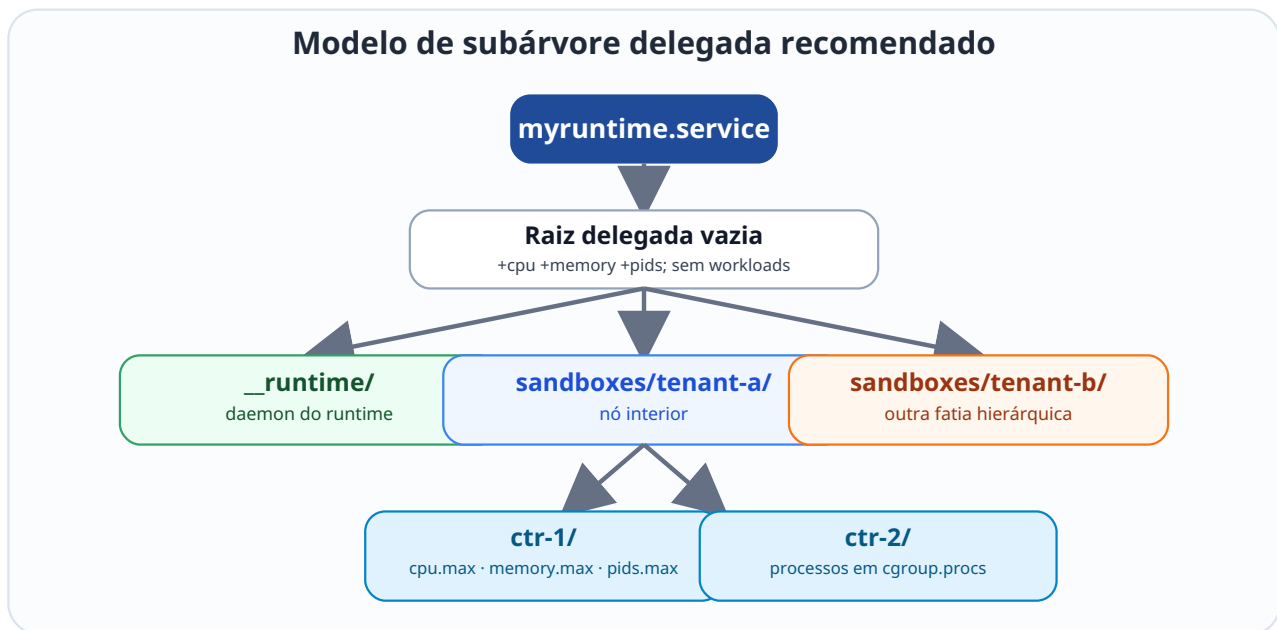


Figura 2 — Exemplo de subárvore delegada: o daemon mora em uma folha interna e os workloads vivem em folhas irmãs.

2.3. “Organize once, control” e por que anexar cedo importa

A man page de cgroups e a própria documentação do kernel deixam claro que a organização deve ser pensada para mudar pouco; migração frequente é custosa e certos tipos de estado não acompanham o processo automaticamente. O caso da memória é particularmente importante: a contabilidade é associada ao cgroup que instanciou a área e não migra simplesmente junto com o processo. Isso significa que o bootstrap do workload deve ser anexado à folha antes que o payload real comece a alocar memória de forma relevante. [2][1]

Essa observação afeta o desenho do runtime inteiro. O data plane não deve fazer *spawn* direto do processo final e só depois tentar movê-lo. O desenho mais robusto cria um *bootstrap child* bloqueado, anexa esse filho à folha via `cgroup.procs` e apenas então libera o `execve()` do payload. Conceitualmente, isso ecoa a abordagem de dois estágios descrita em *libcontainer*, na qual há uma fase de bootstrap separada do init propriamente dito. [10][1]

2.4. Escopo do primeiro corte: domínio, não threaded subtree

Embora o cgroup v2 tenha modos mais sofisticados, inclusive topologias com cgroups *threaded*, o primeiro corte desta feature deveria evitar essa complexidade. A combinação solicitada de limites de CPU, memória e PIDs para containers aponta para envelopes de recurso em nível de workload, não para divisão intra-processo por thread. Ao excluir explicitamente árvores *threaded* na versão inicial, o projeto reduz a superfície de casos inválidos em `cgroup.type` e concentra energia no que interessa: envelopes hierárquicos corretos, attach seguro e recuperação consistente.

3. CPU, memória e PIDs em profundidade

3.1. *cpu.max* e o envelope temporal de CPU

O arquivo *cpu.max* define uma política de banda temporal no formato `MAX PERIOD`. O valor padrão é `max 100000`, o que representa ausência de limite com período de 100 mil microssegundos. Quando se escreve, por exemplo, `150000 100000`, autoriza-se consumo equivalente a 1,5 CPU ao longo de cada janela de 100 ms. A documentação do Docker usa exatamente essa relação ao explicar que `--cpus=1.5` equivale a período de 100000 e quota de 150000. O kernel também ressalta que o controlador de CPU afeta apenas tarefas da classe *fair*; se o runtime expuser suporte a políticas de tempo real no futuro, essa limitação terá de aparecer no contrato do produto. [1][5]

Do ponto de vista de modelagem, isso significa que o control plane não deveria guardar apenas um número decimal “cpus”. Ele deve normalizar a intenção do usuário para um par canônico (`quota_us`, `period_us`), preservando a possibilidade de representar tanto o modo ilimitado quanto quotas precisas. A recomendação para um runtime novo é adotar `period=100000` como padrão porque reduz surpresa, coincide com a semântica já difundida pelo Docker e simplifica comparações em testes de compatibilidade. A API pública pode aceitar `cpus: 1.5`, mas o domínio interno deve operar sobre microssegundos. [5][9]

Na observabilidade, *cpu.stat* é tão importante quanto *cpu.max*. Campos como `nr_periods`, `nr_throttled` e `throttled_usec` dizem se o limite está apenas configurado ou se de fato está causando estrangulamento. Sem isso, o usuário recebe um valor de quota mas não entende o comportamento. Em um Docker-like system bem projetado, métricas e configuração formam um único contrato: o primeiro diz o que o segundo está produzindo na prática. [1][11]

3.2. *memory.max* como limite duro e suas consequências

memory.max é o limite duro do controlador de memória. Quando o uso tenta ultrapassá-lo e não há possibilidade de reclaim suficiente, o cgroup entra no caminho de OOM. A documentação do kernel ainda faz duas observações valiosas. A primeira é que, sob determinadas condições, o limite pode ser excedido temporariamente. A segunda é que, ao diminuir o limite em um cgroup já em uso, o kernel pode fazer reclaim e até acionar OOM no contexto do processo que escreveu, salvo se o arquivo for aberto com `O_NONBLOCK`, caso em que o reclaim síncrono é evitado e a convergência fica diferida para a próxima pressão de memória. [1]

Para a feature em análise, isso separa nitidamente dois caminhos de implementação. No caminho de criação de uma nova folha vazia, escrever *memory.max* é simples, previsível e barato. No caminho de reconfiguração online, reduzir *memory.max* exige política explícita. Um runtime maduro deveria oferecer pelo menos dois modos: um modo estrito, que só aceita a redução se *memory.current* já estiver abaixo do novo teto e portanto evita surpresas operacionais; e um modo diferido, que aceita a nova meta, escreve com semântica não bloqueante e passa a monitorar *memory.events* para detectar reclaim, high pressure e OOM. O erro de produto seria tratar redução de memória como se fosse idêntica a atualização de uma string comum. [1]

Existe ainda uma recomendação prática que merece entrar no desenho do container como valor default. Ao habilitar *memory.oom.group=1* na folha, o kernel passa a considerar o cgroup como unidade de vítima de OOM, evitando o cenário confuso em que uma thread aleatória morre e o processo supervisor do workload continua em estado meio-vivo. Para isolamento de container, a morte coesa do grupo costuma ser preferível porque preserva uma semântica mais legível de falha. [1]

3.3. *pids.max* como barreira contra explosão de tarefas

O controlador de PIDs é menos glamouroso, mas é um dos mais valiosos do ponto de vista de segurança operacional. Em cgroup v2, *pids.max* limita o número de tarefas no grupo, e não apenas o que administradores leigos chamariam de processos. Em aplicações multi-thread, cada thread conta. O kernel documenta que, após migração ou redução do limite, pode ocorrer a situação em que *pids.current* fique maior que *pids.max*; nesse caso, não há morte retroativa das tarefas já existentes, mas novas tentativas de `fork()` ou `clone()` falham com EAGAIN. ^[1]

Essa semântica obriga o control plane a ser didático. A API deve nomear a capacidade corretamente, talvez como “max_tasks” internamente, mesmo que preserve o nome compatível “pids_limit” externamente. Também deve haver validação contextual. Um workload Java, Go ou Nginx multi-thread pode precisar de margens muito maiores que um processo simples em C. O pior desenho é aceitar um limite arbitrário e deixar o usuário descobrir, em produção, que ele limitou threads, não apenas processos. ^[1]

Para o runtime, a boa prática é aplicar *pids.max* antes do primeiro *exec* do payload. Isso fecha cedo a porta para *fork bombs* e evita uma janela de exposição entre criação da folha e início efetivo do workload. Em conjunto com *memory.max* e *cpu.max*, o controlador de PIDs completa o triângulo mínimo de contenção: capacidade de criar trabalho, capacidade de manter memória residente e capacidade de consumir tempo de CPU. ^[1]

3.4. Regras top-down e limites efetivos

Os três controladores só fazem sentido quando lidos hierarquicamente. Mesmo que a folha receba *memory.max=2G*, esse valor é apenas o teto local solicitado. Se um ancestral limitar a subárvore a 1536 MiB, o limite efetivo do workload será menor. O mesmo raciocínio vale para banda de CPU e contagem de tarefas. Em termos de produto, a interface não deveria esconder isso: a resposta da API deveria expor tanto o limite pedido quanto o limite efetivo, depois de aplicado o *clamping* hierárquico. ^[1]

Há uma consequência sutil aqui. O componente que calcula limites efetivos pertence ao domínio e, portanto, ao lado Python da arquitetura. O componente que verifica se a aplicação concreta bate com o kernel pertence ao data plane e, portanto, ao lado C++. Misturar essas duas responsabilidades gera um software difícil de testar. O control plane deve saber responder “qual envelope faz sentido”; o runtime deve saber responder “o kernel agora está coerente com esse envelope”.

Resumo operacional dos três arquivos. *cpu.max* controla banda temporal; *memory.max* controla ocupação de memória com caminho possível de OOM; *pids.max* controla criação de tarefas e retorna EAGAIN quando saturado. O comportamento dos três é hierárquico, e o valor que importa para o usuário é o limite efetivo.

4. Docker original e a tradução para o projeto

4.1. Como o Docker pensa o problema

O Docker não apresenta cgroups ao usuário como um fim em si. Ele apresenta intenção de produto: memória máxima, CPUs disponíveis, número máximo de tarefas, pai de cgroup, classe de runtime. O Engine recebe isso

por CLI ou API, persiste a configuração do container, delega o ciclo de vida ao *containerd* e invoca *runc* como runtime padrão. *libcontainer*, por sua vez, materializa os mecanismos do kernel. Essa divisão é importante porque mostra um princípio replicável: a linguagem da API não precisa coincidir um-para-um com a linguagem dos arquivos do cgroup, desde que exista um modelo canônico no meio. [4][10]

Quando o usuário executa `docker run --memory=512m --cpus=1.5 --pids-limit=256`, ele está pedindo um envelope declarativo. Em um host cgroup v2, esse envelope acaba se convertendo, conceitualmente, em algo como `memory.max=536870912`, `cpu.max=150000 100000` e `pids.max=256`. Essa tradução não precisa ser exposta no contrato público, mas precisa existir de modo determinístico dentro do software. [5][9]

Intenção do usuário	Camada canônica	Expressão final em cgroup v2	Observação de compatibilidade
--memory=512m	OCI LinuxResources.Memory.Limit	memory.max=536870912	No backend systemd, a semântica converge para MemoryMax. [8]
--cpus=1.5	quota/period canônicos	cpu.max=150000 100000	Docker documenta a equivalência com período de 100000 µs. [5]
--pids-limit=256	OCI LinuxResources.Pids.Limit	pids.max=256	No backend systemd, converge para TasksMax. [8]

4.2. cgroupfs, systemd e a noção de pai

O Docker também ensina uma lição prática sobre backends. Em cgroupfs puro, o pai é um caminho na hierarquia; em hosts com driver systemd, o pai passa a ser tipicamente uma *slice* e o leaf final é criado como *scope*, por exemplo terminando em `docker-<id>.scope`. Isso aparece inclusive na documentação do próprio daemon, ao explicar o comportamento de `--cgroup-parent`. Em outras palavras, o mesmo domínio de negócio pode ter duas concretizações mecânicas distintas. [6][8]

No projeto proposto, a consequência é clara. A API e o armazenamento de metadados não devem carregar nomes de arquivos do kernel nem detalhes de systemd como primeira-class citizen. O lado Python precisa falar em termos canônicos como “raiz delegada”, “nó interior”, “folha”, “envelope de recursos” e “backend de governança”. A escolha entre cgroupfs delegado e systemd direto deve ser uma estratégia do runtime, não uma invasão semântica na modelagem de domínio.

4.3. O projeto equivalente em Python, Cython e C++

A analogia mais fiel ao Docker, sem copiá-lo cegamente, seria a seguinte. O processo Python desempenha o papel de um mini Engine: expõe a API, valida entrada, persiste especificações, calcula efeitos hierárquicos, aciona o reconciler e decide que um workload precisa de uma folha específica em uma subárvore específica. Cython funciona como ABI estável entre esse mundo declarativo e a biblioteca nativa. O C++ assume o papel que, no

ecossistema Docker, está espalhado entre runtime e componentes mais próximos do kernel: resolve caminhos, escreve arquivos de cgroup, cria o bootstrap, anexa o PID, coleta métricas e executa cleanup. [4][10]

O ganho dessa arquitetura é que o domínio permanece maleável onde a mudança de produto é frequente e fica rígido onde a integridade operacional é crítica. Alterar políticas de admissão, defaults, cálculo de *effective limits* e contratos da API é muito mais barato em Python. Garantir ordem de syscalls, uso de *pidfd*, manipulação via *dirfd*, sincronização entre *fork* e *exec* e idempotência local é um problema muito mais natural em C++.

Decisão recomendada para o MVP. Em hosts geridos por systemd, o backend inicial deveria ser “cgroupfs dentro de uma subárvore delegada”, não “escrita arbitrária fora da delegação”. Isso respeita a regra do single writer e reduz complexidade de integração. Um backend systemd puro pode entrar depois, atrás de uma Strategy.

5. System Design da feature

5.1. Visão lógica

O sistema deveria ser desenhado como dois planos que cooperam por contrato bem definido. O control plane mantém o estado desejado do mundo. O data plane transforma esse estado desejado em estado materializado no host. Para a feature de *leaf creation*, isso significa que a API recebe um *ContainerSpec* ou *WorkloadSpec*, normaliza o envelope de recursos, resolve onde na hierarquia aquela folha deve existir, persiste uma intenção do tipo “criar leaf X sob o pai Y com limites Z” e então chama a biblioteca nativa por meio do binding Cython. O retorno do data plane inclui o caminho real do cgroup, o identificador do processo bootstrap ou do init, handles de observabilidade e um conjunto claro de erros estruturados.

Esse desenho é compatível tanto com um produto single-node quanto com um produto multi-node. Em modo single-node, o processo Python e a biblioteca nativa residem na mesma máquina e a chamada é in-process. Em modo distribuído, o control plane pode continuar centralizado e existir um agente por host que carrega a extensão Cython. O importante não é a topologia de implantação, e sim a fronteira de responsabilidade: o host é a unidade de verdade operacional para cgroup. Nenhum outro nó do sistema deveria fingir que consegue “aplicar” um limite remotamente sem intermediar um agente local.

5.2. Componentes recomendados

O control plane precisa de, no mínimo, um *API Service*, um *Metadata Repository*, um *Admission and Policy Engine*, um *Scheduler/Placement Resolver* se houver múltiplos hosts, e um *Reconciler*. O data plane precisa de um *Runtime Core* em C++, um *Cgroup Manager*, um *Process Launcher*, um *Metrics Reader* e um pequeno *Failure Journal* ou mecanismo análogo de replay. Em hosts systemd, é recomendável que o serviço do runtime seja declarado com delegação explícita dos controladores que ele poderá administrar. [3][7]

Um detalhe de desenho particularmente importante é a descoberta de capacidades. Nem todo host delegará os mesmos controladores. A documentação do Docker para rootless deixa claro que, por padrão, ambientes rootless com systemd normalmente delegam memória e PIDs; CPU pode exigir configuração adicional de *Delegate=*, e

certos controladores dependem de versões mais novas do systemd. Logo, a capacidade de usar *cpu.max* não deve ser suposta. Ela deve ser detectada em tempo de inicialização e persistida como capacidade do nó. [7]

5.3. Fluxo de criação da folha

O fluxo canônico começa com a validação do pedido. O control plane recebe o spec, converte memória para bytes, CPUs para quota e período, PIDs para inteiro positivo ou ilimitado, e calcula o envelope local e o envelope efetivo com base nos ancestrais conhecidos. Em seguida, cria uma operação persistente do tipo *CreateLeafOperation* com estado *pending*. Só depois dessa persistência o data plane é chamado. Esse ordenamento é importante porque, em caso de crash do processo no meio da operação, o reconciler encontra uma intenção registrada e pode decidir se continua ou faz rollback.

No lado nativo, o runtime primeiro obtém ou reconfirma a raiz delegada, abre a árvore via *dirfd*, garante que o próprio daemon esteja fora do nó interior, assegura que o pai da nova folha esteja vazio de processos e tenha *+cpu*, *+memory* e *+pids* em *cgroup.subtree_control*, cria o diretório da folha, aplica limites na folha vazia, executa o bootstrap child, anexa seu PID a *cgroup.procs* e só então libera o *exec* do payload. O retorno ao Python inclui confirmação do caminho e do PID inicial. [1][10]

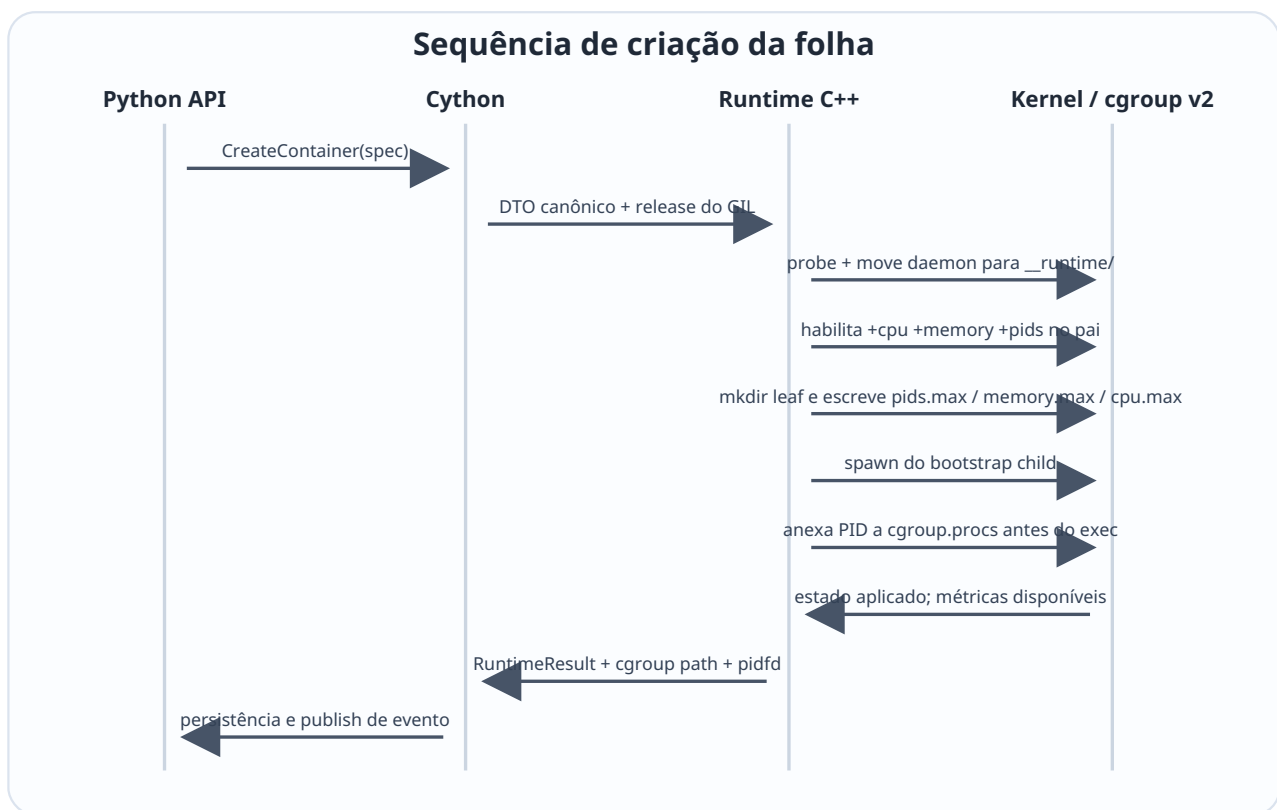


Figura 3 — Sequência recomendada: persistir intenção, materializar folha, anexar PID, liberar exec e publicar estado.

5.4. Observabilidade, cleanup e reconciliação

Depois de criado, o leaf não pode ser tratado como uma pasta esquecida. O runtime deveria coletar ao menos *memory.current*, *memory.events*, *cpu.stat*, *pids.current*, *pids.events* e *cgroup.events*. Isso permite responder a perguntas operacionais elementares: o workload foi limitado? foi estrangulado? entrou em OOM? saturou o

número de tarefas? ainda há processos vivos? O Docker usa control groups também como base para estatísticas de runtime; um projeto do zero precisa honrar a mesma intuição. ^{[1][11]}

Para cleanup, o cgroup v2 oferece ferramentas melhores do que muitos runtimes antigos tinham. Arquivos como *cgroup.events* informam se o grupo ainda está populado, *cgroup.freeze* pode quiescer temporariamente um grupo, e *cgroup.kill* oferece uma forma integrada de terminar processos da subárvore. O desenho correto de teardown é primeiro terminar a execução do workload de modo coerente, depois aguardar a transição para *populated=0* e só então remover a folha vazia. ^[1]

5.5. Uma visão de consistência

Não existe uma transação ACID do kernel que cubra “criar diretório, escrever limites, forkar processo, anexar pid, publicar evento”. Logo, a consistência precisa ser obtida por idempotência e compensação. No plano do domínio, a operação de criação deve ter estados explícitos como *pending*, *preparing*, *leaf_created*, *attached*, *running*, *failed* e *destroyed*. No plano mecânico, cada passo deve ser seguro para repetição. Se a folha já existe e está vazia, o passo “create leaf” é um *load-or-create*; se o PID já está anexado, o attach vira no-op lógico; se o processo morreu, o rollback usa *cgroup.kill* e remoção condicionada pela população do grupo.

6. Low Level Design

6.1. Estruturas de dados no lado Python

No lado Python, vale a pena distinguir três objetos. O primeiro é o *ResourceRequest*, que representa a linguagem externa da API e pode aceitar valores amigáveis como “512m” ou “1.5 CPUs”. O segundo é o *ResourceEnvelope*, uma forma interna canônica e estritamente tipada, por exemplo *memory_max_bytes: int | None*, *cpu_quota_us: int | None*, *cpu_period_us: int* e *pids_max: int | None*. O terceiro é o *EffectiveResourceEnvelope*, já calculado após a aplicação das restrições hierárquicas conhecidas. Essa tripla separa UX, domínio interno e estado operacional.

```
from dataclasses import dataclass

@dataclass(frozen=True)
class ResourceEnvelope:
    memory_max_bytes: int | None
    pids_max: int | None
    cpu_quota_us: int | None
    cpu_period_us: int = 100_000
    oom_group: bool = True

@dataclass(frozen=True)
class LeafPlan:
    container_id: str
    parent_path: str
    envelope: ResourceEnvelope
    backend: str # delegated_cgroupfs | systemd
    expected_controllers: tuple[str, ...] = ("cpu", "memory", "pids")
```

O *LeafPlan* é o objeto que cruza a fronteira entre domínio e execução. Ele não deveria carregar nomes de flags de Docker nem caminhos arbitrários fornecidos pelo usuário. O caminho do pai precisa ser resolvido pelo sistema, a partir de convenções internas e do *placement* decidido. Isso evita path traversal semântico e torna a árvore previsível para reconciliação.

6.2. Fronteira Cython: fina, tipada e com posse de memória explícita

A pior decisão aqui seria transformar Cython em um segundo control plane. O binding deve ser mínimo: receber um *LeafPlan* canônico, convertê-lo para uma struct C++, liberar o GIL durante a operação nativa e traduzir o resultado de volta para um objeto Python. Regras de produto continuam em Python; regras mecânicas de execução continuam em C++.

```
# cython: language_level=3
cdef extern from "runtime_api.hpp" namespace "rt":
    cdef cppclass CgroupSpec:
        unsigned long long memory_max
        long long pids_max
        long long cpu_quota_us
        unsigned long long cpu_period_us
        bint oom_group

    cdef cppclass CreateLeafResult:
        bint ok
        const char* cgroup_path
        int init_pid
        const char* error_code
        const char* error_message

    CreateLeafResult create_leaf(const char* container_id,
                                const char* parent_path,
                                const CgroupSpec& spec) nogil

def create_leaf_py(plan):
    cdef CgroupSpec spec
    # ... preencher struct
    with nogil:
        result = create_leaf(plan.container_id.encode(),
                              plan.parent_path.encode(),
                              spec)
    # ... mapear para exceção ou DTO Python
```

Dois cuidados são essenciais. Primeiro, o binding deve soltar o GIL durante chamadas potencialmente bloqueantes, como escrita em arquivos de controle, *fork*, espera por sincronização e leitura de métricas. Segundo, a tradução de erro precisa preservar códigos de falha estruturados. Um **EINVAL** por topologia inválida, um **EAGAIN** por saturação de PIDs e um erro de política de admissão não podem chegar ao Python todos como um simples “RuntimeError”.

6.3. Objetos centrais no C++

O lado C++ deve ser desenhado em torno de poucos tipos com responsabilidade clara. Um *CapabilityProbe* detecta controladores disponíveis, backend e root atual. Um *CgroupFs* oferece operações baseadas em *dirfd*,

como `mkdirat`, `openat`, `read_file` e `write_file`. Um *SubtreeManager* garante invariantes da hierarquia. Um *ProcessLauncher* faz bootstrap e sincroniza o momento exato do `exec`. Um *StatsReader* lê arquivos de métricas. Um *RecoveryJournal* persiste marcadores locais de operação caso o processo falhe entre passos. Nenhuma dessas classes precisa saber o que é um “tenant plan” ou uma “policy tier”; isso é domínio e fica do outro lado.

6.4. Ordem de operações no algoritmo de criação

A ordem importa. Primeiro, resolver a raiz administrável e validar que o backend pode mesmo aplicar os controladores requisitados. Segundo, garantir que o daemon foi removido do nó interior e está em uma folha interna própria. Terceiro, abrir o pai pretendido por descritores, criar diretórios intermediários se necessário e habilitar controladores em `cgroup.subtree_control` no ancestral correto. Quarto, criar a folha. Quinto, aplicar `pids.max`, `memory.max`, `cpu.max` e `memory.oom.group` enquanto a folha ainda está vazia. Sexto, criar um filho bootstrap bloqueado, escrever seu PID em `cgroup.procs` e apenas então prosseguir com o `exec`. Essa sequência reduz janelas em que o processo poderia executar fora do envelope correto. ^[1]

```
Result Runtime::CreateLeaf(const LeafPlan& plan) {
    auto root = probe_.OpenManagedRoot();
    auto guard = subtree_locks_.Acquire(plan.parent_path);

    EnsureDaemonLivesInInternalLeaf(root);      // __runtime/
    auto parent = EnsurePath(root, plan.parent_path);
    EnsureControllers(parent, {"cpu","memory","pids"});
    EnsureNoProcessesInInterior(parent);

    auto leaf = CreateOrOpenChild(parent, CanonicalLeafName(plan.container_id));
    WriteLimit(leaf, "pids.max", EncodePids(plan.envelope.pids_max));
    WriteLimit(leaf, "memory.max", EncodeBytes(plan.envelope.memory_max_bytes));
    WriteLimit(leaf, "cpu.max", EncodeCpu(plan.envelope.cpu_quota_us,
                                           plan.envelope.cpu_period_us));

    if (plan.envelope.oom_group) {
        WriteLimit(leaf, "memory.oom.group", "1");
    }

    auto child = launcher_.SpawnBootstrap(plan.container_id);
    WritePid(leaf, "cgroup.procs", child.pid);
    launcher_.ReleaseExec(child);

    return Success(leaf.path(), child.pid);
}
```

A presença de `EnsureDaemonLivesInInternalLeaf()` no pseudocódigo é deliberada. Esse método materializa a sutileza mais importante da feature em ambiente `systemd` delegado. Sem ele, o runtime tende a tentar habilitar controladores em um nó ainda populado pelo próprio daemon, batendo nas regras do kernel ou produzindo uma topologia conceitualmente errada. ^{[1][3]}

6.5. Atualização online de limites

Atualização não deve ser tratada como apêndice. Do ponto de vista de produto, o usuário naturalmente vai querer aumentar ou reduzir recursos em um container vivo. Só que os três controladores reagem de formas distintas. Reduzir `cpu.max` é normalmente imediato e resulta em throttling visível em `cpu.stat`. Reduzir `pids.max` abaixo do

consumo atual não mata tarefas existentes; apenas bloqueia novas criações. Reduzir *memory.max* pode bloquear o escritor, disparar reclaim ou induzir OOM. Portanto, o contrato de update precisa ser específico por controlador e a implementação deve preferir métodos distintos de validação e observação para cada caso. ^[1]

Uma boa política é expor, na API, se a atualização é “strict” ou “deferred”. No modo estrito, memória só reduz se *memory.current* já couber; no modo diferido, a intenção é gravada, o valor é tentado de forma não bloqueante quando aplicável, e o reconciler monitora convergência. Para PIDs, o sistema pode aceitar a redução mas deve devolver no resultado que o grupo está temporariamente acima do novo alvo. Esse tipo de transparência evita que o operador confunda “configurado” com “já convergiu”.

6.6. Segurança de filesystem e identificação de processos

Como o runtime escreve em um pseudo-filesystem privilegiado, a implementação deve evitar concatenar strings de caminho de maneira ingênua. O padrão recomendado é trabalhar ancorado em *dirfd* da raiz delegada e usar operações relativas, como *openat* e *mkdirat*, idealmente com restrições adicionais de resolução quando disponíveis. Nomes de folhas devem ser canônicos, derivados de IDs internos, e não diretamente de texto fornecido pela API. Em paralelo, processos devem ser acompanhados por *pidfd* sempre que o kernel permitir, para reduzir risco de reutilização de PID em rotinas de wait, signal e cleanup.

6.7. Recovery e garbage collection

Como o estado de cgroup sobrevive à queda do processo gerenciador, o restart do daemon deve começar por uma reconciliação host-local. A rotina lê a subárvore administrada, enumera folhas, cruza seus nomes com o metadata store, confere *cgroup.events* e decide entre três classes: folhas legítimas em uso, folhas órfãs ainda populadas e folhas vazias removíveis. O erro mais comum em runtimes caseiros é assumir que, se o processo morreu, o kernel “limpou tudo”. Cgroups vazios persistem até remoção explícita; processos vivos podem continuar rodando em folhas órfãs. ^[1]

7. Domain-Driven Design

7.1. Ubiquitous language

Se a equipe pretende sustentar esse runtime por anos, precisa nomear o problema corretamente. Alguns termos deveriam entrar no vocabulário ubíquo desde o início. “Delegated Root” é a raiz local que o runtime pode administrar sem violar o single writer. “Interior Node” é o cgroup cujo papel é organizar a hierarquia e distribuir controladores. “Leaf” é o cgroup terminal que hospeda o workload. “Resource Envelope” é o contrato de memória, CPU e PIDs pedido pelo usuário. “Effective Envelope” é esse contrato depois do *clamping* hierárquico. “Bootstrap Task” é o processo transitório que recebe anexação ao cgroup antes do *exec* do payload. Esses nomes conectam domínio e implementação em vez de escondê-los atrás de expressões vagas.

7.2. Bounded contexts

Para esta feature, pelo menos quatro bounded contexts fazem sentido. O contexto de *Control Plane* cuida de API, autenticação, contratos externos e persistência. O contexto de *Resource Governance* lida com envelopes, topologia de cgroup, limites efetivos e políticas de admissão. O contexto de *Runtime Execution* materializa folhas, lança processos e coleta sinais do kernel. O contexto de *Observability* transforma arquivos de controle e

eventos em métricas, logs estruturados e alertas. O ponto crítico é que o contexto de Resource Governance deve falar com o Runtime Execution por uma *anti-corruption layer* explícita, exatamente o papel desempenhado por Cython e pelo DTO canônico.

7.3. Agregados e invariantes

O agregado mais importante é a *Subtree*. Ele reúne a raiz delegada, os nós interiores, as folhas conhecidas e o conjunto de controladores suportados. Seus invariantes são mais fortes que uma simples árvore de diretórios. Um nó interior que distribui controladores de domínio não pode hospedar processos de workload. Uma folha pode hospedar processos, mas não pode ser simultaneamente tratada como nó interior. Nenhum controlador pode ser exposto em um filho se o pai não o habilitou. O envelope efetivo de uma folha nunca pode exceder o envelope efetivo de seus ancestrais. Tudo isso vem diretamente da semântica do kernel e precisa existir como linguagem de domínio, não apenas como comentários em C++. [1]

Outro agregado útil é o *Container* ou *Workload Instance*. Ele reúne identidade, imagem, processo inicial, caminho do cgroup, envelope local, envelope efetivo e estado de ciclo de vida. Seu estado poderia transitar entre *requested*, *prepared*, *attached*, *running*, *exited*, *reaping* e *destroyed*. A vantagem desse estado explícito é que o reconciler não precisa adivinhar. Ele recebe um objeto que já declara em que fase a instância deveria estar.

7.4. Domain services e repositories

O cálculo de limites efetivos, a validação topológica e a resolução do pai ideal da folha são bons candidatos a *domain services*. Já a persistência de containers, subárvores e operações em andamento deve passar por *repositories*. É útil que o *repository* de operações registre também um *operation journal* curto, porque a criação da folha é um processo multi-etapas. Em uma implementação inicial single-node, SQLite com WAL pode ser suficiente; em um ambiente distribuído, um banco central e um *journal* local por host podem coexistir. O importante é que a escolha de armazenamento não altere os invariantes do domínio.

7.5. Domain events

Eventos de domínio merecem atenção especial porque ajudam a costurar observabilidade e reconciliação. Alguns exemplos são *LeafPlanned*, *LeafPrepared*, *BootstrapAttached*, *ContainerRunning*, *ContainerOOMKilled*, *PidsLimitHit* e *LeafGarbageCollected*. Eventos assim não são meros logs. Eles reduzem o acoplamento entre o Runtime Execution e o Observability Context, além de permitirem replay e auditoria. O gatilho bruto pode vir de arquivos como *memory.events*, *pids.events* e *cgroup.events*; a interpretação disso em linguagem de produto, porém, é papel do domínio. [1]

8. Padrões de Design aplicáveis

8.1. Composite para representar a árvore

Entre os padrões GoF, o mais natural aqui é *Composite*. A hierarquia de cgroups é literalmente uma árvore composta por nós que podem conter outros nós. Representar *DelegatedRoot*, *InteriorNode* e *LeafNode* sob uma abstração comum permite percorrer, validar e serializar a topologia sem espalhar condicionais por todo o código. O padrão é particularmente útil no control plane, onde cálculos de envelope efetivo e validação de top-down dependem da navegação pela árvore.

8.2. Strategy e Abstract Factory para backends

O runtime deveria encapsular a materialização de recursos atrás de uma *Strategy* de backend. A implementação primária do MVP pode ser *DelegatedCgroupFsStrategy*, que escreve diretamente na subárvore delegada. Mais tarde, uma *SystemdStrategy* pode criar slices e scopes via D-Bus, traduzindo o mesmo envelope canônico para os objetos de systemd. Uma *Abstract Factory* pode instanciar a estratégia correta com base em capacidades detectadas no host, sem vaziar essa decisão para a API pública. [3][8]

8.3. Template Method para o pipeline de criação

O pipeline *prepare* → *create leaf* → *apply limits* → *spawn bootstrap* → *attach* → *exec* → *publish* é quase invariável, mesmo que os detalhes mudem por backend. Isso o torna um candidato excelente a *Template Method*. A classe base define o fluxo e pontos de extensão, enquanto subclasses variam o modo de resolver o pai, escrever limites ou integrar-se com systemd. O ganho é enorme na testabilidade: os testes do algoritmo principal ficam estáveis, e apenas os *hooks* específicos precisam de variações por backend.

8.4. Command para ações operacionais e retries

Criar uma folha, atualizar limites, congelar um grupo, matar um grupo e destruí-lo são ações naturalmente modeláveis como *Command*. Cada comando pode carregar seus argumentos, seu identificador idempotente e um método de compensação. Isso simplifica retries, journaling e reconciliação. Em um projeto distribuído, o mesmo padrão ainda facilita serialização e rastreabilidade, porque o pedido persistido se aproxima bastante da ação real executada no host.

8.5. State para o ciclo de vida do container

O padrão *State* ajuda a conter a explosão de condicionais em torno do ciclo de vida do workload. Um container *requested* aceita preparação, mas não coleta métricas. Um container *attached* já possui PID e cgroup, mas talvez ainda não tenha completado *exec*. Um container *running* aceita atualização de limites e leitura de métricas. Um container *exited* não aceita mais *exec*, mas ainda pode demandar limpeza. Essa modelagem melhora a clareza do domínio e reduz estados impossíveis.

8.6. Adapter e Facade na fronteira Cython

Cython cumpre naturalmente o papel de *Adapter* ao converter o mundo Python para o mundo C++ e vice-versa. Também é útil enxergá-lo como uma pequena *Facade* que esconde detalhes de ABI, gerenciamento de memória e códigos de erro. O resultado ideal é que o resto do código Python veja apenas métodos como `create_leaf(plan)`, `update_leaf_limits(plan)` e `destroy_leaf(id)`, sem precisar saber se por baixo existe `cgroupfs`, `systemd`, `pidfd` ou detalhes de `openat`.

8.7. Observer para métricas e eventos

Por fim, o padrão *Observer* encaixa-se bem no subsistema de observabilidade. O runtime lê eventos do kernel e publica sinais sem impor acoplamento ao restante do sistema. O contexto de *Observability* assina esses eventos e os transforma em séries temporais, logs ou alertas. A beleza desse desenho é que o kernel permanece fonte de fatos brutos, enquanto o domínio interpreta o significado desses fatos.

9. Requisitos funcionais e não funcionais

9.1. Requisitos funcionais

RF-01 — A implementação deve criar folhas válidas em cgroup v2

O sistema precisa ser capaz de criar um leaf abaixo de um pai previamente resolvido, habilitando em tempo oportuno `cpu`, `memory` e `pids` no `cgroup.subtree_control` do ancestral correto, e garantindo que nós interiores não abriguem processos de workload. A criação deve ser idempotente: repetir a operação não pode produzir duplicidade lógica nem corromper a subárvore. ^[1]

RF-02 — A implementação deve aplicar *memory.max*, *pids.max* e *cpu.max* sobre folhas vazias antes do exec

O runtime precisa materializar os limites em uma ordem segura, ainda com a folha vazia, de forma que o payload nunca comece a executar fora do envelope desejado. Sempre que possível, o bootstrap do processo deve ser anexado à folha antes da primeira alocação significativa de memória. ^[1]

RF-03 — A implementação deve expor *requested* e *effective limits*

O control plane não pode devolver apenas os valores pedidos pelo usuário. Ele deve calcular e expor também os limites efetivos, depois do clamping imposto pelos ancestrais da árvore. Isso é obrigatório para tornar inteligível a regra top-down e evitar surpresas operacionais.

RF-04 — A implementação deve suportar atualização online de limites

CPU, memória e PIDs precisam poder ser alterados em runtime, com semântica explícita por controlador. O retorno da operação deve informar não apenas se a nova configuração foi aceita, mas também se ela já convergiu ou se ainda há divergência temporária, como no caso de *pids.current* acima de um novo teto ou de uma redução de memória em modo diferido. ^[1]

RF-05 — A implementação deve observar o kernel e publicar eventos de domínio

Métricas mínimas incluem *memory.current*, *memory.events*, *cpu.stat*, *pids.current*, *pids.events* e *cgroup.events*. A partir desses fatos brutos, o sistema deve emitir eventos de domínio como OOM do container, saturação de PIDs, throttling de CPU e esvaziamento de um leaf para coleta de lixo. ^[1]

RF-06 — A implementação deve realizar cleanup coerente

Ao destruir um container, o runtime precisa encerrar ou matar o grupo, aguardar a subárvore ficar não populada e então remover a folha. O caminho de destruição precisa ser seguro mesmo se o processo gerenciador tiver reiniciado no meio de uma operação anterior. ^[1]

RF-07 — A implementação deve detectar capacidades do host e falhar de forma explícita

Se um host rootless só delega memória e PIDs, a tentativa de impor *cpu.max* não pode ser silenciosamente ignorada. O sistema deve anunciar quais controladores estão de fato disponíveis e rejeitar ou degradar pedidos de maneira explícita e auditável. ^[7]

RF-08 — A implementação deve suportar ao menos um backend delegado e prever outro backend

O produto mínimo viável deveria funcionar sobre cgroupfs dentro de uma subárvore delegada. Ao mesmo tempo, a arquitetura deve deixar clara a possibilidade de um backend systemd. Isso é importante para não aprisionar o domínio na mecânica do MVP. ^{[3][8]}

RF-09 — A implementação deve reconciliar estado após reinicialização

Na subida do agente, a subárvore administrada deve ser varrida e comparada com o metadata store. Folhas órfãs, containers ainda vivos e cgroups vazios precisam ser detectados automaticamente. Sem isso, o runtime acumula drift a cada queda de processo.

RF-10 — A implementação deve impor limites de crescimento da própria árvore

É recomendável usar controles como *cgroup.max.descendants* e *cgroup.max.depth* onde fizer sentido, para impedir explosão acidental ou maliciosa de subárvores. Em um sistema que aceitará pedidos externos, essa proteção não é luxo; é higiene de plataforma. ^[1]

9.2. Requisitos não funcionais

RNF-01 — Correção topológica

A topologia observável no kernel deve ser sempre compatível com as regras do cgroup v2. Não é aceitável obter sucesso aparente na API enquanto a árvore real viola pressupostos de domínio, como nós interiores populados ou controladores não delegados. ^[1]

RNF-02 — Idempotência operacional

Toda operação materializada no host precisa ser segura para repetição. Isso é a base de retries, reconciler e tolerância a falhas entre persistência e syscall. A ausência de idempotência transforma qualquer queda de processo em corrupção operacional.

RNF-03 — Segurança de escrita no pseudo-filesystem

A implementação deve evitar concatenação textual ingênua de caminhos, trabalhar por descritores sempre que possível, gerar nomes canônicos internamente e manter uma única raiz delegada claramente definida. Isso reduz risco de bugs de path traversal, colisão de nomes e escrita fora do domínio administrável.

RNF-04 — Observabilidade suficiente para explicar comportamento

O objetivo não é apenas medir, mas explicar. O sistema deve conseguir responder por que um workload ficou lento, por que não conseguiu criar threads, por que foi morto por OOM ou por que um cgroup não foi removido. Sem essa capacidade, o runtime se torna uma caixa-preta hostil em produção.

RNF-05 — Baixa latência de criação

A criação da folha, excluídos tempos de download de imagem e preparação de filesystem, deve ser rápida o bastante para não dominar o tempo de start do container. Em termos práticos, um alvo inicial razoável é manter o trecho “materializar leaf + anexar bootstrap” em ordem de poucos milissegundos a poucas dezenas de milissegundos por container em hosts modernos, desde que não haja contenção externa anormal.

RNF-06 — Compatibilidade operacional com `hosts systemd`

Em `hosts` com `systemd`, o runtime não deve violar a regra do `single writer`. Toda escrita fora de uma subárvore delegada ou sem backend apropriado deve ser tratada como configuração inválida, não como atalho aceitável. ^[3]

RNF-07 — Clareza semântica da API

O contrato da API precisa dizer claramente que PIDs contam tarefas, que memória pode OOM-killar o grupo, que CPU é modelada como quota e período, e que limites efetivos podem ser menores que limites pedidos. Ambiguidade semântica na API é dívida técnica, não gentileza.

RNF-08 — Testabilidade realista

Testes unitários são necessários, mas insuficientes. A feature deve ser coberta por testes de integração em kernel real, cenários de stress e fault injection. Não existe confiança legítima em um runtime de `cgroup` construída só com mocks.

10. Roadmap de implementação e testes

10.1. Fase 0 — Bootstrap do ambiente e `capability probe`

A primeira entrega não deveria criar containers ainda. Ela deveria apenas descobrir o host: verificar montagem de `cgroup v2`, controladores disponíveis, presença de `systemd`, existência de delegação, caminho da raiz administrável, permissões de escrita e possibilidade de coletar métricas básicas. Nessa fase, o daemon também já deve implementar a migração para a folha interna `__runtime/` quando estiver operando sob raiz delegada. A maior parte dos bugs conceituais aparece aqui, não na etapa final.

10.2. Fase 1 — Criação estrita de leaf vazio e `attach` do bootstrap

A segunda entrega implementa o caminho feliz mínimo: criar a subárvore, habilitar controladores, criar a folha, aplicar `pids.max`, `memory.max` e `cpu.max`, lançar um bootstrap child e anexá-lo antes do `exec`. Ainda sem update online sofisticado, mas com persistência de operações e cleanup básico. O foco aqui é provar a correção topológica e a ordem de operações.

10.3. Fase 2 — Observabilidade e `update online`

Com a criação básica estabilizada, entram leitura de `cpu.stat`, `memory.current`, `memory.events`, `pids.current`, `pids.events` e `cgroup.events`, além do suporte a atualização online com semântica explícita por controlador. Essa é a fase em que o runtime deixa de ser apenas um executor de syscalls e passa a ser uma plataforma observável.

10.4. Fase 3 — Reconciler robusto e `garbage collection`

Na terceira fase, o restart do agente deve ser tratado como fluxo comum, não excepcional. O reconciler precisa varrer a subárvore, classificar folhas legítimas, órfãs e vazias, publicar eventos de reparo e coletar lixo com segurança. Também é nessa fase que vale introduzir `cgroup.kill` como caminho preferencial de teardown e limites defensivos para profundidade e descendentes. ^[1]

10.5. Fase 4 — Backend systemd e hardening

Somente depois do backend delegado estar sólido faz sentido adicionar um backend systemd puro, se ele realmente trouxer valor ao produto. Essa fase inclui também endurecimento de segurança, uso amplo de *pidfd*, melhoria de mensagens de erro, testes de concorrência e suporte mais refinado a ambientes rootless. [8][7]

10.6. Estratégia de testes

O conjunto de testes precisa ser desenhado para reproduzir semânticas do kernel, não apenas caminhos de código. Um teste essencial é o de “daemon ainda no nó interior”: o sistema deve falhar ou auto-corrigir movendo o próprio processo para a folha interna. Outro teste indispensável é o de memória já acima do novo teto em update, para verificar diferença entre modos estrito e diferido. Há ainda os testes de *fork bomb* controlado para validar *pids.max*, de CPU burner para observar throttling em *cpu.stat*, de limites hierárquicos contraditórios para validar o cálculo de effective limits e de reconciliação após crash deliberado do runtime.

Ordem de prioridade sugerida. Primeiro, corrigir topologia. Segundo, anexar processos no momento certo. Terceiro, tornar observável. Quarto, aceitar updates online. Quinto, adicionar backends alternativos. Runtimes falham mais por errar a base do que por demorar uma iteração para ganhar refinamentos.

11. Conclusão

Construir *cgroup v2 leaf creation* em um Docker-like system do zero não é um exercício de escrever três strings em um pseudo-filesystem. É um problema de engenharia de domínio e de engenharia de sistemas ao mesmo tempo. O domínio precisa modelar envelopes de recursos, topologia de subárvore, requested versus effective limits, capacidades do host e ciclo de vida do workload. O sistema precisa respeitar a semântica do kernel, a delegação do systemd, a ordem correta de attach e a recuperação após falha. Quando essas duas camadas são misturadas, o software fica frágil; quando são separadas com rigor, o resultado é uma base extensível.

A recomendação principal deste estudo é adotar uma arquitetura em que Python detenha o controle semântico do produto e C++ detenha a execução concreta no host, com Cython reduzido a uma camada adaptadora, fina e estável. O primeiro backend deveria ser *cgroupfs* dentro de uma subárvore delegada, justamente para respeitar a regra do single writer e simplificar o MVP em hosts systemd. O algoritmo de criação deve ser orientado por uma sequência inviolável: preparar a raiz, mover o daemon para uma folha interna, habilitar controladores no pai, criar a folha, aplicar limites em uma folha vazia, anexar o bootstrap child e só então liberar o payload. [1][3]

Também vale reforçar três decisões que, embora pequenas, têm impacto grande. A primeira é expor limites efetivos na API, e não apenas limites pedidos. A segunda é tornar a semântica de update online explícita por controlador, especialmente para memória. A terceira é coletar métricas e eventos desde a primeira versão útil do runtime, porque sem observabilidade o operador não entende por que o kernel respondeu da forma que respondeu. [1][11]

Se essa feature for implementada com o nível de rigor descrito aqui, ela deixa de ser uma engrenagem isolada e passa a ser o esqueleto do runtime inteiro. A partir dela, namespaces, filesystem, rede, exec, pause, stats e

autoscaling passam a herdar uma base de isolamento já coerente. Em outras palavras: acertar a folha em cgroup v2 é acertar o coração do sistema.

Referências

- [1] Linux Kernel Documentation. *Control Group v2*. Documentação oficial da hierarquia unificada, controladores, eventos, memória, CPU, PIDs, congelamento e kill. Disponível em: kernel.org/doc/html/latest/admin-guide/cgroup-v2.html.
- [2] Michael Kerrisk. *cgroups(7)*. Manual page do Linux, incluindo recomendações sobre organização da árvore e controle de recursos. Disponível em: man7.org/linux/man-pages/man7/cgroups.7.html.
- [3] systemd. *Control Group APIs and Delegation*. Documentação do systemd sobre a regra do single writer, unidades, delegation e integração com gerentes de containers. Disponível em: systemd.io/CGROUP_DELEGATION/.
- [4] Docker Inc. *Docker overview*. Visão geral do Docker Engine, daemon, API, CLI e arquitetura cliente-servidor. Disponível em: docs.docker.com/get-started/docker-overview/.
- [5] Docker Inc. *Resource constraints*. Documentação de limites de memória, CPU e PIDs, incluindo a equivalência de `--cpus` para quota e período. Disponível em: docs.docker.com/engine/containers/resource_constraints/.
- [6] Docker Inc. *dockerd reference*. Referência do daemon, inclusive semântica de `--cgroup-parent` em cgroupfs e systemd. Disponível em: docs.docker.com/reference/cli/dockerd/.
- [7] Docker Inc. *Rootless mode tips*. Observações sobre cgroup v2, systemd e delegação de controladores em modo rootless. Disponível em: docs.docker.com/engine/security/rootless/tips/.
- [8] opencontainers/runc. *systemd driver*. Documentação do driver systemd, uso de scopes e tradução de recursos para ambientes cgroup v2. Disponível em: github.com/opencontainers/runc/blob/main/docs/systemd.md.
- [9] Open Container Initiative. *Runtime Specification — config-linux*. Modelo canônico de recursos Linux usado na tradução para runtimes. Disponível em: github.com/opencontainers/runtime-spec/blob/main/config-linux.md.
- [10] opencontainers/runc. *libcontainer README*. Descrição do bootstrap em dois estágios e integração com mecanismos do kernel. Disponível em: github.com/opencontainers/runc/tree/main/libcontainer.
- [11] Docker Inc. *Runtime metrics*. Uso de cgroups como base para estatísticas e introspecção de runtime. Disponível em: docs.docker.com/engine/containers/runmetrics/.